线性分类模型

数据科学与大数据技术专业

第 3 讲

1 分类问题与线性模型

1. **分类任务的基本特征**

   - 分类任务的目标是将输入映射到离散标签空间, 输出属于有限个类别之一.
   - 与回归相比, 分类问题更关心决策边界和类别可分性; 在实际应用中, 决策边界往往依赖非线性特征表示.
   - 典型应用包括图像识别、文本分类、医疗诊断和风险预警等.

2. **线性分类模型的判别方式**

   - 线性分类器通常先计算线性得分

     $$f_c(\boldsymbol{x}) = \boldsymbol{w}_c^\top \boldsymbol{x} + b_c,$$

     再根据得分输出类别.
   - 在二分类中, 一个超平面即可将特征空间划分为两个半空间.
   - 在多分类中, 常见策略包括:
     - **一对多 (One-vs-Rest, OvR)**: 为每个类别训练一个二分类器, 预测时选择得分最高的类别.
     - **一对一 (One-vs-One, OvO)**: 为任意两类分别训练一个二分类器, 共需 $\frac{C(C-1)}{2}$ 个分类器, 预测时通过投票决定类别.
     - **直接 $\arg\max$ 判别**: 同时为 C 个类别建立得分函数, 将 $\arg\max_c f_c(\boldsymbol{x})$ 作为预测结果.

3. **分类评价指标**

   - 给定测试集 $\{(\boldsymbol{x}^{(n)}, y^{(n)})\}_{n=1}^N$ 和预测结果 $\hat{y}^{(n)}$, 准确率与错误率分别为

     $$\text{Accuracy} = \frac{1}{N}\sum_{n=1}^{N} \mathbb{I}\big(y^{(n)} = \hat{y}^{(n)}\big), \qquad \text{ErrorRate} = 1 - \text{Accuracy}.$$

   - 在二分类中, 混淆矩阵刻画了正负类上的局部表现:

	预测为正类 ($\hat{y} = 1$)	预测为负类 ($\hat{y} = 0$)
真实为正类 ($y = 1$)	真正例 (True Positive, TP)	假负例 (False Negative, FN)
真实为负类 ($y = 0$)	假正例 (False Positive, FP)	真负例 (True Negative, TN)

- 基于混淆矩阵, 常用指标包括:
  - **精确率 (Precision)**:
$$\text{Precision} = \frac{TP}{TP + FP}.$$
    它衡量所有"预测为正"的样本中有多少确为正类.
  - **召回率 (Recall)** 或真正例率 (True Positive Rate, TPR):
$$\text{Recall} = \text{TPR} = \frac{TP}{TP + FN}.$$
    它衡量所有真实正类样本中有多少被模型成功识别.
  - F_1 **值**: 精确率与召回率的调和平均,
$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}.$$
  - F_β **值**: 用于调节对精确率和召回率的偏好,
$$F_\beta = \frac{(1+\beta^2)\text{PrecisionRecall}}{\beta^2\text{Precision} + \text{Recall}}, \qquad \beta > 0.$$
    其中 $\beta > 1$ 时更偏重召回率, $0 < \beta < 1$ 时更偏重精确率.
- **ROC 曲线与 AUC**: 对输出为概率或打分的二分类器, 可通过调整阈值来考察不同工作点下的性能.
  - 在阈值 t 下,
$$\text{TPR}(t) = \mathbb{P}\big(s(\boldsymbol{x}) \geq t \mid y = 1\big), \qquad \text{FPR}(t) = \mathbb{P}\big(s(\boldsymbol{x}) \geq t \mid y = 0\big).$$
    以 FPR 为横轴、TPR 为纵轴即可得到 ROC 曲线.
  - ROC 曲线越靠近左上角 $(0, 1)$, 模型区分能力通常越强; 若曲线接近对角线 $y = x$, 则模型与随机猜测接近.
  - 若正负样本的打分分布相同, 则对任意阈值都有 $\text{TPR}(t) = \text{FPR}(t)$, 因而 ROC 曲线退化为对角线.
  - AUC 是 ROC 曲线下的面积,
$$\text{AUC} = \int_0^1 \text{TPR}(u)\,\mathrm{d}u, \qquad u = \text{FPR}.$$
    它衡量模型在所有阈值下的整体排序能力.
  - 更准确地说, AUC 还可解释为
$$\text{AUC} = \mathbb{P}\big(s(x^+) > s(x^-)\big) + \frac{1}{2}\mathbb{P}\big(s(x^+) = s(x^-)\big),$$
    即随机抽取一个正样本和一个负样本时, 正样本得分不低于负样本的概率.

4. **典型线性分类器**

- **Logistic 回归**: 用于二分类概率建模.
- **Softmax 回归**: Logistic 回归在多分类问题上的自然推广.

- **感知器**: 基于错误驱动的在线学习算法.
- **支持向量机 (SVM)**: 通过最大化间隔提升分类鲁棒性.

问题思考:

- 线性模型如何实现多分类?
- 感知器与 Logistic 回归的主要区别是什么?

2 信息论与损失函数

1. **从极大似然到交叉熵**

   - 在分类问题中, 模型通常输出条件概率分布 $P(y \mid \boldsymbol{x}; \boldsymbol{w})$, 而不是直接输出类别标签.
   - 给定训练集 $\{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^{n}$, 极大似然估计的目标是

   $$\max_{\boldsymbol{w}} \prod_{i=1}^{n} P(y^{(i)} \mid \boldsymbol{x}^{(i)}; \boldsymbol{w}),$$

   等价地, 也可最大化对数似然

   $$\max_{\boldsymbol{w}} \sum_{i=1}^{n} \log P(y^{(i)} \mid \boldsymbol{x}^{(i)}; \boldsymbol{w}).$$

   - 因此, 训练分类模型通常等价于最小化负对数似然 (Negative Log-Likelihood, NLL), 这正是交叉熵损失的来源.

2. **二分类中的交叉熵**

   - 设 $y \in \{0, 1\}$, 并记模型输出的正类概率为

   $$p(\boldsymbol{x}) = P(y = 1 \mid \boldsymbol{x}; \boldsymbol{w}).$$

   则条件概率可统一写为

   $$P(y \mid \boldsymbol{x}; \boldsymbol{w}) = p(\boldsymbol{x})^{y} \big(1 - p(\boldsymbol{x})\big)^{1-y}.$$

   - 对独立同分布样本取负对数似然, 得到经验风险

   $$\mathcal{L}(\boldsymbol{w}) = -\sum_{i=1}^{n} \log P(y^{(i)} \mid \boldsymbol{x}^{(i)}; \boldsymbol{w}) = -\sum_{i=1}^{n} \Big[y^{(i)} \log p(\boldsymbol{x}^{(i)}) + \big(1 - y^{(i)}\big) \log\big(1 - p(\boldsymbol{x}^{(i)})\big) \Big].$$

   - 若将真实标签看作分布 $[y, 1-y]$, 将模型输出看作分布 $[p(\boldsymbol{x}), 1 - p(\boldsymbol{x})]$, 则上式恰好等于这两个分布的交叉熵. 因此, 在分类问题中最小化 NLL 与最小化交叉熵是一回事.

3. **信息熵, 交叉熵与 KL 散度**

   - 对离散分布 $p(x)$, 信息熵定义为

   $$H(p) = -\sum_{x} p(x) \log p(x).$$

- 若用预测分布 $q(x)$ 去近似真实分布 $p(x)$, 则交叉熵定义为

$$H(p,q) = -\sum_x p(x)\log q(x).$$

- KL 散度定义为

$$\mathrm{KL}(p\|q) = \sum_x p(x)\log\frac{p(x)}{q(x)} = H(p,q) - H(p).$$

  因为 $H(p)$ 与模型参数无关, 所以最小化交叉熵与最小化 $\mathrm{KL}(p\|q)$ 等价.

- **KL 散度的非负性**: 由不等式

$$\log t \le t-1, \qquad \forall t>0,$$

  令 $t = \frac{q(x)}{p(x)}$, 则对所有满足 $p(x)>0$ 的 x 有

$$\log\frac{q(x)}{p(x)} \le \frac{q(x)}{p(x)} - 1.$$

  两边乘以 $p(x)$ 并对 x 求和, 得到

$$\sum_x p(x)\log\frac{q(x)}{p(x)} \le \sum_x \big(q(x)-p(x)\big) = 0.$$

  因而

$$\mathrm{KL}(p\|q) = \sum_x p(x)\log\frac{p(x)}{q(x)} \ge 0.$$

  等号成立当且仅当 $p(x)=q(x)$ 在所有 $p(x)>0$ 的位置上成立.

问题思考:

- 为什么交叉熵损失适合分类问题?
- KL 散度与交叉熵的关系是什么?

3 Logistic 回归 (二分类)

1. **模型形式与概率解释**

   - 为了书写简洁, 以下默认将偏置项并入特征向量. 于是模型先计算线性得分 $\boldsymbol{w}^\top\boldsymbol{x}$, 再通过 Sigmoid 函数输出正类概率.
   - Logistic 函数定义为

$$\sigma(z) = \frac{1}{1+\exp(-z)}.$$

     因而

$$p(y=1 \mid \boldsymbol{x};\boldsymbol{w}) = \hat{y} = \sigma(\boldsymbol{w}^\top\boldsymbol{x}).$$

   - 预测时, 常将 $\hat{y}$ 与阈值 0.5 比较, 或直接输出概率作为置信度.

2. **损失函数与经验风险**

- Logistic 回归通常采用二元交叉熵作为损失函数.
- 在包含 N 个样本的训练集上, 平均经验风险写为

$$\mathcal{R}(\boldsymbol{w}) = -\frac{1}{N}\sum_{n=1}^{N}\left[y^{(n)}\log\hat{y}^{(n)} + \left(1-y^{(n)}\right)\log\left(1-\hat{y}^{(n)}\right)\right].$$

3. **梯度计算与参数更新**

- 由 Sigmoid 函数的导数

$$\sigma'(z) = \sigma(z)\big(1-\sigma(z)\big),$$

可得经验风险关于参数的梯度为

$$\begin{aligned}\frac{\partial\mathcal{R}(\boldsymbol{w})}{\partial\boldsymbol{w}} &= -\frac{1}{N}\sum_{n=1}^{N}\left(\frac{y^{(n)}}{\hat{y}^{(n)}} - \frac{1-y^{(n)}}{1-\hat{y}^{(n)}}\right)\hat{y}^{(n)}\big(1-\hat{y}^{(n)}\big)\boldsymbol{x}^{(n)} \\ &= \frac{1}{N}\sum_{n=1}^{N}\big(\hat{y}^{(n)} - y^{(n)}\big)\boldsymbol{x}^{(n)}.\end{aligned}$$

- 若采用学习率为 α 的梯度下降法, 则参数更新为

$$\boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t - \alpha\frac{1}{N}\sum_{n=1}^{N}\big(\hat{y}_{\boldsymbol{w}_t}^{(n)} - y^{(n)}\big)\boldsymbol{x}^{(n)}.$$

4. **训练特性与停止策略**

- Logistic 回归会持续根据"预测概率与真实标签之间的残差"调整参数, 即使样本已经被分对, 只要模型仍不够自信, 参数仍会继续更新.
- 在线性可分且不加正则化时, Logistic 损失往往没有有限的最优参数, 优化过程会倾向于不断增大 $\|\boldsymbol{w}\|$ 来把概率推向 0 或 1.
- 因此, 实际训练中常结合 L_2 正则化、早停策略或验证集监控, 以改善泛化能力与数值稳定性.

问题思考:

- 为什么 Logistic 回归输出概率?
- 交叉熵损失如何影响模型训练?

4 Softmax 回归 (多分类)

1. **从二分类到多分类**

- Softmax 回归是 Logistic 回归在多分类问题上的自然推广.
- 对每个类别 $c \in \{1, \ldots, C\}$, 模型先计算线性得分

$$z_c = \boldsymbol{w}_c^\top\boldsymbol{x} + b_c,$$

再通过归一化映射得到类别概率.

- 预测时通常选择最大概率对应的类别, 即 $\arg\max_c \hat{y}_c$.

2. **Softmax 概率映射**

   - 对得分向量 $\boldsymbol{z} = (z_1, \ldots, z_C)$, Softmax 定义为

$$\hat{y}_c = \mathrm{softmax}(z_c) = \frac{\exp(z_c)}{\sum_{j=1}^{C} \exp(z_j)}, \qquad c = 1, \ldots, C.$$

   - 该映射保证每个 $\hat{y}_c > 0$, 且 $\sum_{c=1}^{C} \hat{y}_c = 1$, 因而输出可以解释为类别条件概率分布.

3. **损失函数与模型优化**

   - 若第 n 个样本的真实标签采用 one-hot 向量 $\boldsymbol{y}^{(n)} = (y_1^{(n)}, \ldots, y_C^{(n)})$, 模型预测为 $\hat{\boldsymbol{y}}^{(n)} = (\hat{y}_1^{(n)}, \ldots, \hat{y}_C^{(n)})$, 则多元交叉熵经验风险可写为

$$\mathcal{R} = -\frac{1}{N} \sum_{n=1}^{N} \sum_{c=1}^{C} y_c^{(n)} \log \hat{y}_c^{(n)}.$$

   - 当 Softmax 与交叉熵结合使用时, 对每个样本的得分分量 $z_c^{(n)}$ 有

$$\frac{\partial \mathcal{R}}{\partial z_c^{(n)}} = \hat{y}_c^{(n)} - y_c^{(n)},$$

   这一梯度形式非常简洁, 因而适合用梯度下降及其变种进行优化.

问题思考:

- Softmax 回归与 Logistic 回归的联系与区别是什么?
- 多分类问题中, Softmax 函数的核心作用是什么?

4.1 补充: 为什么使用 Softmax

1. **最大熵视角**

   - 设得分向量 $x = (x_1, \ldots, x_C) \in \mathbb{R}^C$, 希望构造概率分布 $p = (p_1, \ldots, p_C)$, 满足 $p_i > 0$, $\sum_{i=1}^{C} p_i = 1$, 并满足期望约束

$$\sum_{i=1}^{C} p_i x_i = \mu.$$

   - 在仅保留这些已知信息的前提下, 最大熵原则要求求解

$$\max_p \left\{ -\sum_{i=1}^{C} p_i \log p_i \right\} \quad \text{s.t.} \quad \sum_{i=1}^{C} p_i = 1, \qquad \sum_{i=1}^{C} p_i x_i = \mu.$$

   - 引入拉格朗日乘子后, 可得解的形式为

$$p_i = \frac{\exp(\lambda x_i)}{\sum_{j=1}^{C} \exp(\lambda x_j)}.$$

   当 $\lambda = 1$ 时, 就得到标准的 Softmax 形式.

2. **平移不变性与数值稳定性**

- 对任意常数 $c \in \mathbb{R}$, 有

$$\mathrm{softmax}(x_i + c) = \frac{\exp(x_i + c)}{\sum_{j=1}^{C} \exp(x_j + c)} = \frac{\exp(x_i)}{\sum_{j=1}^{C} \exp(x_j)}.$$

  因而

$$\mathrm{softmax}(\boldsymbol{x} + c\mathbf{1}) = \mathrm{softmax}(\boldsymbol{x}).$$

- 这表明 Softmax 只依赖各类别得分之间的相对差异. 实际计算中, 常先减去 $\max_i x_i$ 来避免指数溢出.

3. **从 Logistic 到 Softmax**

- 在二分类中, Logistic 回归通过 Sigmoid 函数把一个实数得分映射为正类概率.
- 在多分类中, 我们需要把 C 个类别的得分同时映射为和为 1 的概率分布. 对每个得分取指数并统一归一化, 就得到 Softmax.
- 因而, Softmax 可以看作 Logistic 回归在多分类情形下的直接推广.

4. **与交叉熵的配合**

- 设原始得分为 $\boldsymbol{z} = (z_1, \ldots, z_C)$, 预测概率为

$$\hat{y}_k = \frac{\exp(z_k)}{\sum_{j=1}^{C} \exp(z_j)}, \qquad k = 1, \ldots, C.$$

  对应的交叉熵损失为

$$\mathcal{L} = -\sum_{k=1}^{C} y_k \log \hat{y}_k.$$

- 由

$$\log \hat{y}_k = z_k - \log \sum_{j=1}^{C} \exp(z_j),$$

  可得

$$\frac{\partial \mathcal{L}}{\partial z_m} = \hat{y}_m - y_m.$$

  这一结果说明梯度恰好等于“预测概率减去真实概率”, 因而推导和实现都非常直接.

5. **物理视角**

- Softmax 的形式与统计物理中的玻尔兹曼分布一致:

$$P(E_i) = \frac{\exp\left(-\frac{E_i}{kT}\right)}{\sum_j \exp\left(-\frac{E_j}{kT}\right)}.$$

- 若把类别得分看成负能量, 或把温度参数吸收到得分缩放中, 就得到与 Softmax 相同的形式. 这说明 Softmax 也可以理解为一种“得分越高, 概率越大”的能量模型分布.

6. **历史补充**

- Softmax 这一名称通常追溯到 John S. Bridle 在 1989 年关于分类网络输出概率解释的工作 [1]. 该工作明确将归一化指数映射用于多分类神经网络输出层, 也促成了 Softmax 在机器学习中的广泛使用.

5 感知器 (Perceptron)

1. **模型定义**

   - 感知器提出于 1958 年, 是早期机器学习中的经典线性分类模型.
   - 对二分类标签 $y \in \{+1, -1\}$, 感知器使用线性判别函数并通过符号函数作出预测:

   $$\hat{y} = \mathrm{sign}(\boldsymbol{w}^\top \boldsymbol{x}).$$

2. **错误驱动的更新规则**

   - 感知器的训练目标是找到一组参数, 使所有样本满足

   $$y^{(n)}\big(\boldsymbol{w}^\top \boldsymbol{x}^{(n)}\big) > 0.$$

   - 当样本 $(\boldsymbol{x}, y)$ 被分错, 即

   $$y\big(\boldsymbol{w}^\top \boldsymbol{x}\big) \le 0,$$

   就执行更新

   $$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t + y\boldsymbol{x}.$$

   - 这一更新可看作对感知器损失

   $$L_{\mathrm{perc}}(\boldsymbol{w}; \boldsymbol{x}, y) = \max\big(0, -y\boldsymbol{w}^\top \boldsymbol{x}\big)$$

   的一次随机次梯度下降. 这里使用的是感知器损失, 而不是带单位间隔的 Hinge 损失.

3. **为何只对错分样本更新**

   - 若样本已经满足 $y\boldsymbol{w}^\top \boldsymbol{x} > 0$, 则其感知器损失为 0, 不会推动参数继续变化.
   - 若样本被分错, 则更新后有

   $$y\big(\boldsymbol{w}_{t+1}^\top \boldsymbol{x}\big) = y\big(\boldsymbol{w}_t^\top \boldsymbol{x}\big) + y^2 \boldsymbol{x}^\top \boldsymbol{x},$$

   其中第二项恒非负, 因而该样本的分类裕度会朝正确方向增大.

4. **收敛性与局限**

   - 若训练数据线性可分, 感知器算法会在有限步内收敛到某个可分解.
   - 若数据本身不可线性分, 标准感知器通常不会收敛, 参数可能持续振荡.

5. **与 Logistic 回归的比较**

   - 感知器只关心“是否分对”, 一旦样本处在正确一侧就停止更新.

- Logistic 回归则基于概率残差更新参数, 即使样本已经分对, 只要预测概率与真实标签仍有偏差, 参数就可能继续调整.

问题思考:

- 感知器算法为何只对分错样本更新?
- 感知器收敛性的前提条件是什么?

6 支持向量机 (SVM)

1. **最大间隔思想**
   - 虽然许多线性分类器都能找到把两类样本分开的超平面, 但不同超平面的鲁棒性并不相同.
   - 支持向量机的核心思想是在所有可分超平面中寻找间隔最大的那个, 以提升模型对扰动的稳定性.
   - 与最优超平面距离最近的样本称为**支持向量**, 它们决定了最终的决策边界.
2. **硬间隔 SVM**
   - 设分类超平面为 $\boldsymbol{w}^\top \boldsymbol{x} + b = 0$, 对样本 $(\boldsymbol{x}^{(n)}, y^{(n)})$ 其中 $y^{(n)} \in \{+1, -1\}$, 其几何间隔可写为
   $$\gamma^{(n)} = \frac{y^{(n)}\big(\boldsymbol{w}^\top \boldsymbol{x}^{(n)} + b\big)}{\|\boldsymbol{w}\|}.$$
   - 若数据严格线性可分, 可要求所有样本满足 $y^{(n)}(\boldsymbol{w}^\top \boldsymbol{x}^{(n)} + b) \geq 1$, 并求解
   $$\begin{array}{ll} \min_{\boldsymbol{w},b} & \frac{1}{2}\|\boldsymbol{w}\|^2 \\ \text{s.t.} & y^{(n)}\big(\boldsymbol{w}^\top \boldsymbol{x}^{(n)} + b\big) \geq 1, \quad n = 1, \ldots, N. \end{array}$$
   - 这一形式通过固定函数间隔为 1 消除了缩放不确定性, 等价于最大化最小几何间隔.
3. **软间隔 SVM**
   - 实际数据往往含有噪声, 甚至本身不可线性分. 此时可以引入松弛变量 $\xi_n \geq 0$, 允许部分样本落入间隔内部或被错分.
   - 软间隔 SVM 的优化问题为
   $$\begin{array}{ll} \min_{\boldsymbol{w},b,\boldsymbol{\xi}} & \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_{n=1}^{N} \xi_n \\ \text{s.t.} & y^{(n)}\big(\boldsymbol{w}^\top \boldsymbol{x}^{(n)} + b\big) \geq 1 - \xi_n, \quad n = 1, \ldots, N, \\ & \xi_n \geq 0, \quad n = 1, \ldots, N. \end{array}$$
   - 当 $0 < \xi_n < 1$ 时, 样本虽然仍被分对, 但已经落入间隔内部; 当 $\xi_n \geq 1$ 时, 样本可能被错分.
   - 超参数 $C > 0$ 用于平衡"最大化间隔"和"惩罚违约"这两个目标: C 越大, 对违约样本惩罚越强; C 越小, 模型越强调间隔和鲁棒性.
4. **Hinge 损失视角**

- 在适当重参数化下, 软间隔 SVM 等价于最小化 Hinge 损失与正则化项之和:

$$\min_{\boldsymbol{w},b}\sum_{n=1}^{N}\max\Big(0,1-y^{(n)}\big(\boldsymbol{w}^\top\boldsymbol{x}^{(n)}+b\big)\Big)+\frac{\lambda}{2}\|\boldsymbol{w}\|^2.$$

- Hinge 损失只要分类裕度未超过 1 就继续惩罚, 因而不仅要求"分对", 还要求"分得足够远".

5. **与其他分类损失的比较**

- 若记 $m=yf(\boldsymbol{x})$ 为分类裕度, 则不同损失函数对大裕度样本的处理方式不同.
- **平方损失**: $\mathcal{L}=(1-m)^2$. 它对已经正确且裕度很大的样本仍会持续惩罚, 因而通常不如专门的分类损失自然.
- **Logistic 损失**: $\mathcal{L}=\log(1+\exp(-m))$. 该损失光滑、便于优化, 且具有概率解释.
- **感知器损失**: $\mathcal{L}=\max(0,-m)$. 只要样本分对, 损失就为零.
- **Hinge 损失**: $\mathcal{L}=\max(0,1-m)$. 它比感知器损失更强调安全间隔, 因而更有利于提升鲁棒性.

6. **优化方法**

- 经典 SVM 常通过对偶问题转化为凸二次规划来求解, 代表算法包括 SMO(Sequential Minimal Optimization).
- 在大规模问题中, 也可以采用随机次梯度下降等近似优化方法.

问题思考:

- SVM 为何要最大化间隔?
- 软间隔 SVM 如何处理噪声数据?

7 模型对比总结

1. **模型选择的主要考虑**

- 若需要直接输出二分类概率, 通常选择 Logistic 回归.
- 若需要多分类概率输出, 通常选择 Softmax 回归.
- 若数据线性可分且关注在线更新, 感知器是最简单的基线方法.
- 若更关注分类间隔与鲁棒性, 则 SVM 往往更有优势.

2. **典型模型对比**

模型	激活函数	损失函数	优化方法	主要应用场景
线性回归	-	均方误差	最小二乘/梯度下降	回归任务
Logistic 回归	Sigmoid	二元交叉熵	梯度下降	二分类概率预测
Softmax 回归	Softmax	多元交叉熵	梯度下降	多分类概率预测
感知器	符号函数 (sgn)	感知器损失	在线更新/随机梯度法	线性可分分类
SVM	符号函数 (sgn)	Hinge 损失 + L_2 正则	二次规划/SMO/次梯度法	最大间隔分类

问题思考:

- 不同模型的损失函数有何异同?
- 线性模型在实际应用中应如何选择?

(注: 详细定义和更多内容请参考课程教材第 3 章及附录 E.)

参考文献

[1] John S. Bridle. Probabilistic interpretation of feedforward classification network outputs, with relationships to statistical pattern recognition. In *Neurocomputing: Algorithms, Architectures and Applications*, pages 227–236. Springer Berlin Heidelberg, Berlin, Heidelberg, 1990.